

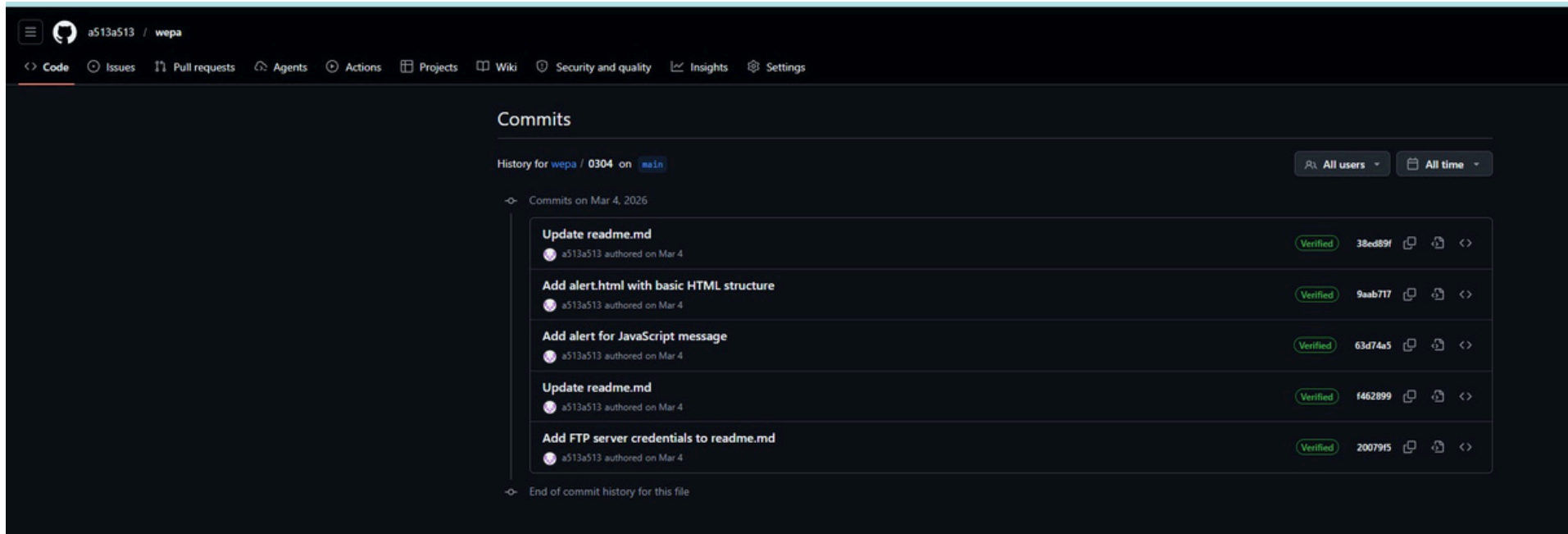
웹 프로그래밍

깃허브 정리

<https://github.com/a513a513/wepa>

2021963034 박준홍

1주차



vs code code snippet에 대해 설명해줘

VS Code: 코드 스니펫 (Code Snippet) 🚀 정의 및 목적 코드 스니펫은 반복적으로 사용되는 코드 패턴을 줄임말(Trigger) 하나로 즉시 불러오는 자동 완성 기능입니다.

“단순한 타이핑 도구를 넘어, 개발의 생산성과 일관성을 높이는 필수 기능입니다.”

- ◆ 주요 장점 생산성 극대화: 수십 줄의 보일러플레이트(Boilerplate) 코드를 단 몇 글자로 입력하여 개발 시간을 단축합니다.

오류 방지: 복잡한 문법 구조를 미리 템플릿화하여 오타로 인한 런타임 에러를 방지합니다.

표준화: 팀 단위 프로젝트에서 공통된 코드 스타일(컨벤션)을 유지하기 용이합니다.

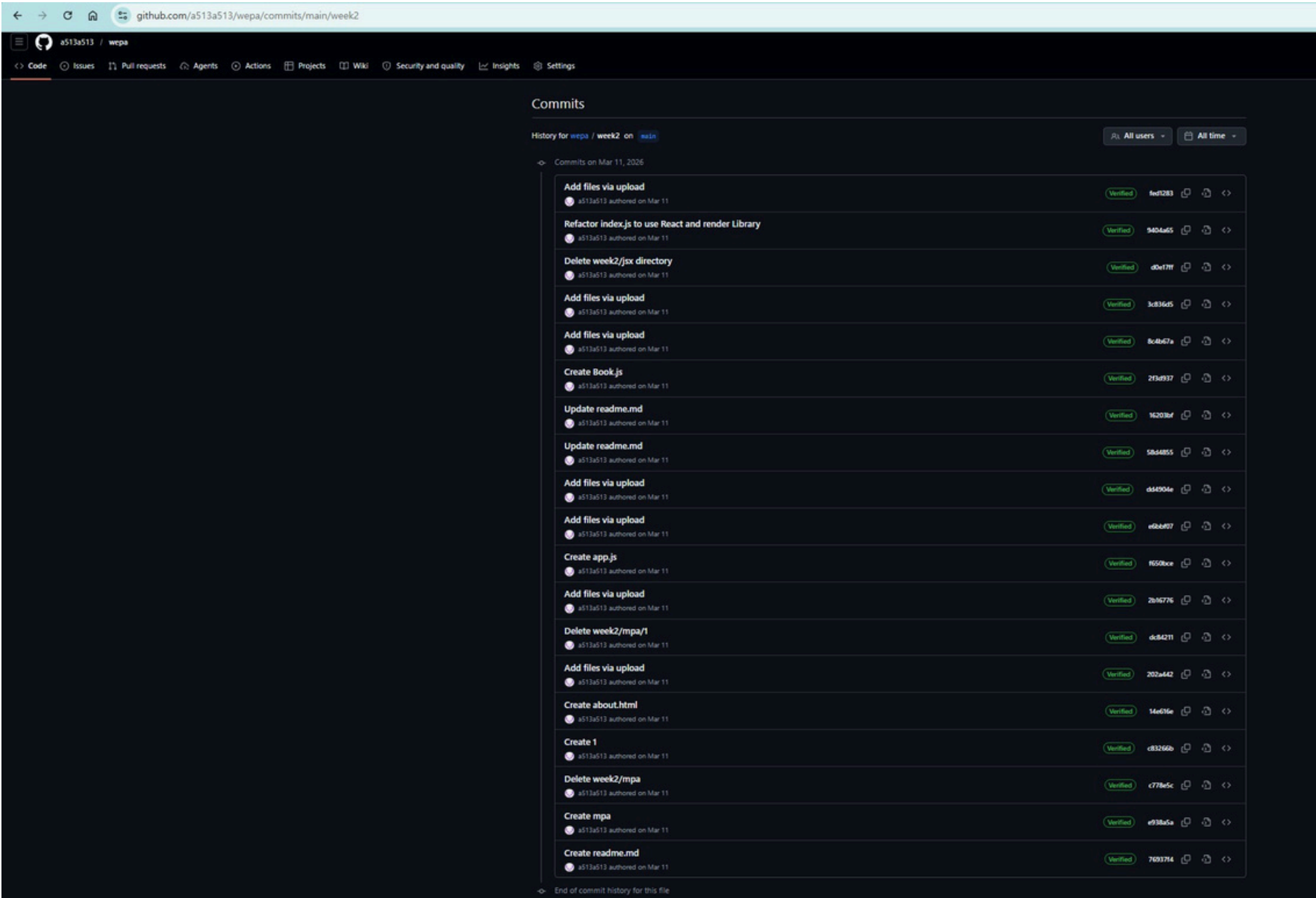
🔥 사용자 정의 스니펫 예시 (JSON) 사용자가 직접 sout이라는 줄임말을 설정하여 자바의 출력문을 자동 완성하는 예시입니다.

JSON { "Java System Out": { "prefix": "sout", "body": ["System.out.println(\$1);"], "description": "콘솔 출력을 위한 System.out.println() 스니펫" } }
prefix: 스니펫을 실행할 키워드 (입력창에 sout 입력)

body: 실제 삽입될 코드 내용 (\$1은 자동 완성 후 커서가 위치할 지점)

description: 해당 스니펫에 대한 설명 -제미나이

2주차



웹 프로그래밍 요약: HTML/CSS 및 React 기초

1. HTML & CSS 및 웹 기초

마크업 언어 vs 메타데이터

- 마크업 언어: 문서나 데이터를 구조화하고 서식을 지정하는 언어입니다 (예: HTML, XML, Markdown). 주로 콘텐츠의 표형에 초점을 맞춥니다.
- 메타데이터: 데이터 자체가 아닌, 데이터를 설명하는 정보입니다 (예: HTML의 `<meta>` 태그, JSON-LD). 마크업 언어는 문서 내에 메타데이터를 포함할 수 있습니다.

HTML 표준 기관

- W3C 웹 표준 제정 기관으로 HTML의 공식 명세를 제공합니다.
- MDN Web Docs: 태그 목록과 사용법을 상세히 설명하며, 최신 표준을 쉽게 이해할 수 있습니다.
- WHATWG: 최신 HTML Living Standard(HTML LS)를 유지보수하는 단체입니다.

MPA vs SPA

- MPA (Multi-Page App): 매 요청마다 서버에서 새로운 HTML을 받아 페이지 전체를 새로고침합니다.
- SPA (Single-Page App): 초기 요청에만 HTML을 받고, 이후에는 AJAX를 통해 JSON 데이터만 받아 화면의 필요한 부분만 갱신합니다.

Node.js

- 서버 측에서 자바스크립트를 실행할 수 있게 해주는 런타임 환경(Environment)입니다.
- OS나 웹 프레임워크가 아니며, 특정 운영체제에 종속되지 않고 실행 환경을 제공합니다.

개발 도구

- IDE: 코드 편집, 디버깅, 빌드 등을 하나로 통합한 도구입니다 (예: Visual Studio, IntelliJ IDEA).
- 코드 편집기: 가볍고 빠른 성능을 제공하며, 특히 VS Code는 크로스 플랫폼과 풍부한 확장 기능을 지원하여 널리 쓰입니다.

2. React와 CRA (Create React App)

npm vs npx

- npm: Node.js 패키지를 설치하고 의존성을 관리하는 패키지 매니저입니다.
- npx: 패키지를 로컬에 설치하지 않고 일회성으로 바로 실행하는 도구입니다.

Vite vs Webpack

구분	Vite	Webpack
개발 서버 속도	빠름 (ESM 기반)	느림 (변동형 필요)
HMR 지원	즉각적 반영	상대적으로 느림
변동형 방식	필요할 때만 빌드	전체 파일 변동형

리액트 프로젝트 생성 명령어

- Vite 사용 (권장): `npm create vite@latest 프로젝트명 --template react` 후 `npm install` 및 `npm run dev` 실행.
- CRA 사용: `npx create-react-app 프로젝트명` 후 `npm start` 실행.

3. JSX (JavaScript XML)

- JavaScript 코드 안에서 HTML과 유사한 마크업을 사용할 수 있게 해주는 문법 확장입니다.
- Babel을 이용해 `React.createElement()` 형태의 JavaScript로 변환됩니다.
- 장점: 코드가 직관적이고 가독성이 높으며, 조건부 렌더링 및 이벤트 핸들링 구현이 쉽습니다.

JSX 주요 문법 및 사용법

- 기본 구조: 반드시 하나의 부모 요소(예: `div` 또는 `xs`)로 감싸야 합니다.
- 표현식: 자바스크립트 변수나 표현식은 `{}` 로 감싸서 사용합니다.
- 속성(Props): HTML의 `class` 대신 `className` 을 사용하고, 인라인 스타일은 `style={{ 속성: "값" }}` 형태의 객체로 전달합니다.
- 조건부 렌더링: 삼항 연산자(조건 ? 참 : 거짓) 또는 `aa` 연산자를 활용합니다.
- 반복문: 배열의 `map()` 메서드를 사용하며, 최상위 반복 요소에는 고유한 `key` 속성이 필수입니다.
- 이벤트 처리: `onClick`, `onChange` 등 카멜 케이스(CamelCase)로 작성합니다.

3주차

a513a513 / wepa

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Commits

History for wepa / week3 on `main`

All usersAll time

<- Commits on Mar 18, 2026

Create index.js

a513a513 authored on Mar 18

Verified13b8263

<>

Add Library component to display book list

a513a513 authored on Mar 18

Verified4e79fb5

<>

Delete week3/JavaScript

a513a513 authored on Mar 18

Verified923d929

<>

Add Library component to display books

a513a513 authored on Mar 18

Verified2ef409f

<>

Add Book component to display book details

a513a513 authored on Mar 18

Verified522a5ca

<>

Update readme.md

a513a513 authored on Mar 18

Verified4aa19e1

<>

Remove citation references from readme.md

a513a513 authored on Mar 18

Verified6511ead

<>

Create readme.md

a513a513 authored on Mar 18

Verifiede8de6b8

<>

<- End of commit history for this file

Web Programming 01

1. HTML & CSS

HTML 문서구조

HTML 문서의 구조는 다음과 같다. <!doctype> 선언, <html> 루트 태그, <head> 머리 태그 영역, <body> 본문 영역, </html> 종료 태그. <head>에는 메타데이터, 스타일시트, 링크 등이 포함된다. <body>에는 웹페이지의 본문 내용이 포함된다.

HTML 문서의 구조를 시각적으로 표현한 그림이다. <html> 루트 태그 아래에는 <head>와 <body>가 있다. <head>에는 <meta>, <title>, <link>, <script> 등이 포함된다. <body>에는 웹페이지의 본문 내용이 포함된다.

2. JavaScript & DOM

DOM(Document Object Model)

DOM은 웹문서의 구조를 표현하는 API이다. DOM은 웹문서의 구조를 트리 구조로 표현한다. DOM은 웹문서의 구조를 트리 구조로 표현한다. DOM은 웹문서의 구조를 트리 구조로 표현한다.

DOM은 웹문서의 구조를 트리 구조로 표현한다. DOM은 웹문서의 구조를 트리 구조로 표현한다. DOM은 웹문서의 구조를 트리 구조로 표현한다.

3. 웹 렌더링 과정 (Rendering)

Rendering

렌더링은 웹문서를 브라우저에서 표시하는 과정이다. 렌더링은 웹문서의 구조를 트리 구조로 표현한다. 렌더링은 웹문서의 구조를 트리 구조로 표현한다. 렌더링은 웹문서의 구조를 트리 구조로 표현한다.

렌더링은 웹문서의 구조를 트리 구조로 표현한다. 렌더링은 웹문서의 구조를 트리 구조로 표현한다. 렌더링은 웹문서의 구조를 트리 구조로 표현한다.

4. SPA vs MPA

SPA vs MPA

SPA(Single Page Application)와 MPA(Multi Page Application)는 웹 애플리케이션의 두 가지 유형이다. SPA는 클라이언트 측에서 렌더링을 수행한다. MPA는 서버 측에서 렌더링을 수행한다.

SPA와 MPA의 차이점을 비교한 그림이다. SPA는 클라이언트 측에서 렌더링을 수행한다. MPA는 서버 측에서 렌더링을 수행한다.

5. Client - Server

Client - Server

클라이언트와 서버 간의 통신을 나타내는 그림이다. 클라이언트는 서버에게 데이터를 요청하고, 서버는 클라이언트에게 데이터를 응답한다.

클라이언트와 서버 간의 통신을 나타내는 그림이다. 클라이언트는 서버에게 데이터를 요청하고, 서버는 클라이언트에게 데이터를 응답한다.

6. React 기초

React 기초

React는 웹 애플리케이션을 구축하기 위한 라이브러리이다. React는 컴포넌트 기반의 아키텍처를 제공한다. React는 컴포넌트 기반의 아키텍처를 제공한다.

React는 컴포넌트 기반의 아키텍처를 제공한다. React는 컴포넌트 기반의 아키텍처를 제공한다. React는 컴포넌트 기반의 아키텍처를 제공한다.

7. JSX (JavaScript XML)

JSX (JavaScript XML)

JSX는 JavaScript와 XML을 결합한 문법이다. JSX는 HTML 요소를 JavaScript 코드에 작성할 수 있게 해준다. JSX는 HTML 요소를 JavaScript 코드에 작성할 수 있게 해준다.

JSX는 HTML 요소를 JavaScript 코드에 작성할 수 있게 해준다. JSX는 HTML 요소를 JavaScript 코드에 작성할 수 있게 해준다. JSX는 HTML 요소를 JavaScript 코드에 작성할 수 있게 해준다.

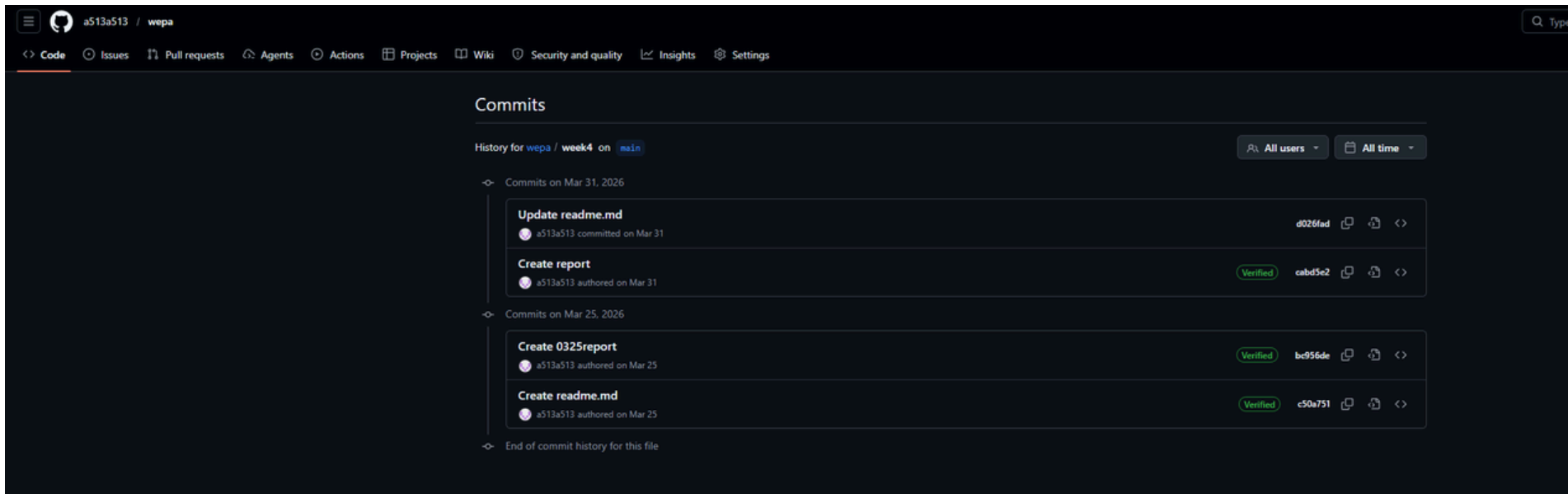
8. JSX 컴포넌트 문법 규칙

JSX 컴포넌트 문법 규칙

JSX 컴포넌트 문법 규칙은 다음과 같다. JSX 컴포넌트는 하나의 JSX 요소를 반환해야 한다. JSX 컴포넌트는 하나의 JSX 요소를 반환해야 한다.

JSX 컴포넌트 문법 규칙은 다음과 같다. JSX 컴포넌트는 하나의 JSX 요소를 반환해야 한다. JSX 컴포넌트는 하나의 JSX 요소를 반환해야 한다.

4 주차



1. Element (엘리먼트) 개념: 웹 페이지 구조를 이루는 기본 단위 (예: <태그>는 엘리먼트, 그 안의 내용은 콘텐츠).

React Element: 리액트 앱을 구성하는 가장 작은 블록. JSX 문법을 사용해 HTML 태그처럼 작성합니다.

2. Component (컴포넌트) 개념: 화면을 나누는 재사용 가능한 독립적 단위 (엘리먼트들을 그룹화한 것).

역할: HTML에서 UI를 나누는 구역(Header, Content 등)과 같은 역할을 리액트에서 수행합니다.

종류: Function Component(함수형), Class Component(클래스형)

3. Function Component (함수형 컴포넌트) 개념: 함수를 사용해 JSX를 반환(return)하여 UI를 구성하는 방식.

주의점: 반환 시 반드시 하나의 최상위 부모 태그로 감싸야 합니다.

✗ 잘못된 예시: return

Hello

Hi

; (루트 요소가 2개)

✓ 올바른 예시: return

Hello

Hi

;

4. Class Component (클래스형 컴포넌트) 개념: 함수 대신 class 문법을 사용하여 만드는 컴포넌트.

특징: 반드시 render() 메서드 안에서 JSX를 반환해야 하며, 함수형과 마찬가지로 두 개 이상의 루트 요소가 올 수 없습니다. (<> 같은 Fragment 사용 가능)

5. React 렌더링 및 변화 감지 순서 초기 렌더링: 컴포넌트가 렌더링되어 가상 DOM(Virtual DOM)이 생성되고, 이를 기반으로 실제 DOM이 브라우저에 표시됩니다.

상태 변화: 컴포넌트의 상태(State)나 속성(Props)이 변경되면 새로운 가상 DOM이 생성됩니다.

비교 (Diffing): 이전 가상 DOM과 새로운 가상 DOM을 비교해 어느 부분이 변경되었는지 찾아냅니다.

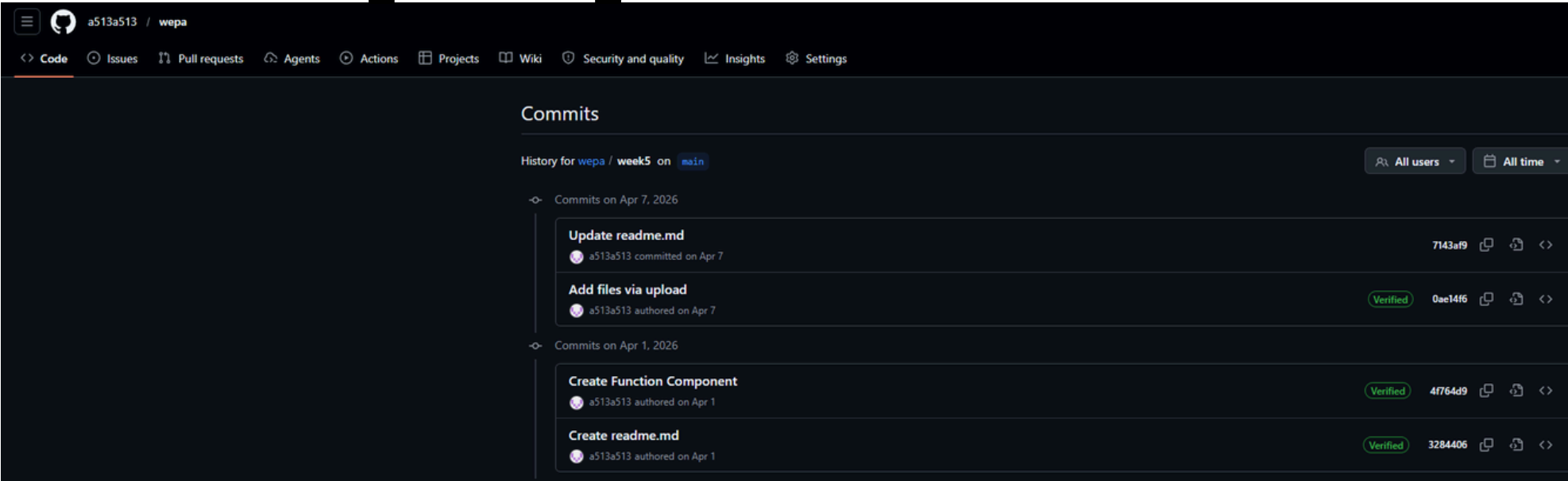
업데이트: 변경이 확인된 부분(과 하위 객체들)만 실제 DOM에 최소한으로 업데이트하여 성능을 최적화합니다.

6. Root Node (루트 노드) 개념: 모든 컴포넌트를 담고 있는 리액트 트리의 최상위 객체.

동작: index.html에 있는

를 시작점으로 하여, 부모에서 자식 방향으로 컴포넌트를 렌더링합니다.

5주차



1. Component (컴포넌트)의 역할 개념: 입력(Props)을 받아 출력(Element)을 반환하는 리액트 앱의 기본 단위입니다.

특징: 리액트는 컴포넌트 기반 구조이며, 태그들의 집합인 컴포넌트들이 모여 하나의 전체 화면을 구성합니다.

주의 사항: * 컴포넌트 이름은 반드시 대문자로 시작해야 합니다.

소문자로 시작하면 리액트가 이를 일반 HTML DOM 태그로 인식하여 오류가 발생하거나 원치 않는 디자인이 적용될 수 있습니다.

2. Props (프로퍼티) 개념: 상위(부모) 컴포넌트가 하위(자식) 컴포넌트에 데이터를 전달할 때 사용하는 수단입니다.

특징: * 단방향 데이터 흐름: 데이터는 항상 부모에서 자식으로만 흐릅니다.

읽기 전용 (Immutable): 하위 컴포넌트는 받은 Props를 직접 수정할 수 없습니다.

3. Props 사용 예시 [App.js] (부모 컴포넌트) JavaScript import React from 'react'; import MyComponent from './MyComponent';

```
function App() { return (  
  /* 하위 컴포넌트에 name이라는 props로 각각 다른 값을 전달 */  
); }
```

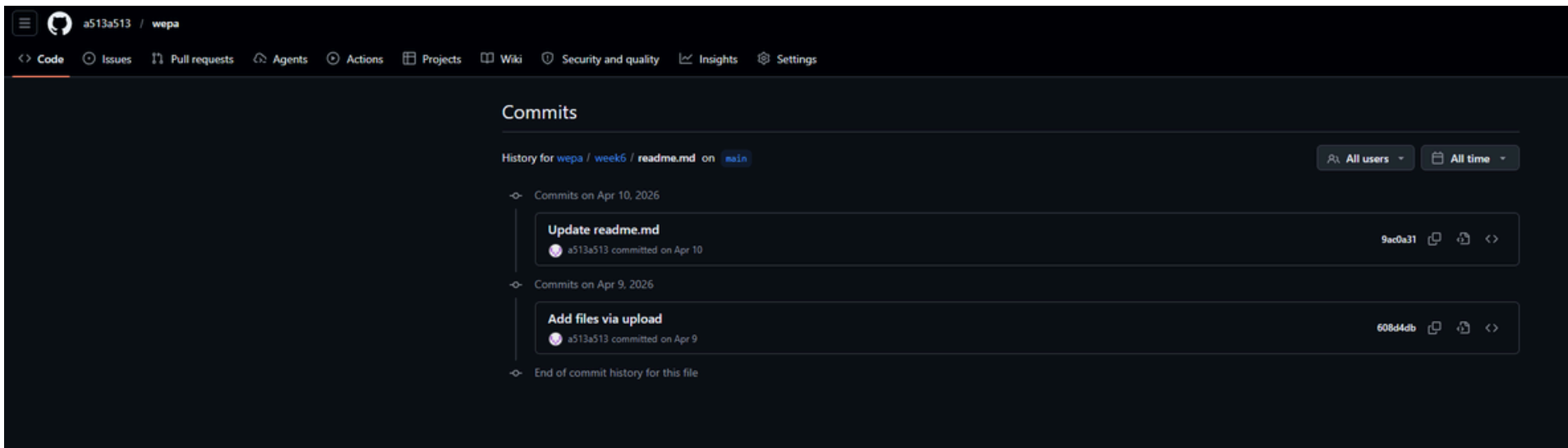
export default App; [MyComponent.js] (자식 컴포넌트) JavaScript import React from 'react';

```
const MyComponent = (props) => { /* 부모에게 전달받은 props.name을 출력 */} return
```

```
{props.name}로 만드는 테스트 페이지  
; };
```

export default MyComponent; 컴포넌트 계층 구조 (데이터 흐름) Plaintext [최상위] index.html → index.js → App.js (부모) → MyComponent (자식) [최하위]

6 주차



개발 블로그에 올리는 스터디 노트나, 같이 공부하는 동료가 정리해서 공유해 준 느낌으로 문체를 조금 바꿔봤습니다.

🔥 리액트 핵심: State(상태) 짚고 넘어가기 State는 컴포넌트의 화면 렌더링에 직접적인 영향을 주는 데이터예요. 이 State 값이 바뀌면 리액트가 알아서 해당 컴포넌트를 다시 그려줍니다(재렌더링). 시계 앱에서 1초마다 시간이 바뀌면 화면이 자동으로 갱신되는 걸 떠올리면 이해하기 쉽습니다.

가장 주의해야 할 점은 State 값을 변수 바꾸듯 직접 수정하면 안 된다는 거예요. (예: `state.value = 1` ❌) 이렇게 하면 리액트가 값이 변한 걸 눈치채지 못 해서 화면이 업데이트되지 않거든요. 유일한 예외는 클래스형 컴포넌트의 `constructor`(생성자)에서 처음 초기값을 세팅할 때뿐입니다.

🔥 컴포넌트 방식에 따른 State 관리법

1. 클래스형 컴포넌트 (Class Component) 클래스형에서는 `this.state`로 상태를 만들고, 값을 바꿀 때는 무조건 `this.setState()`를 사용해야 합니다.

비동기 처리 & 성능 최적화: `setState`는 비동기적으로 동작해요. 즉, 짧은 시간에 여러 번 호출되더라도 리액트가 이걸 모아서 한 번에 처리합니다. 불필요하게 화면을 여러 번 다시 그리는 걸 막아주죠.

주의점: 비동기 방식이라 값을 바꾼 직후에 바로 그 값을 확인하기 어려울 수 있어요. 그래서 이전 상태 값을 기반으로 새로운 값을 계산해야 할 때는 객체 대신 함수를 전달하는 방식이 훨씬 안전합니다.

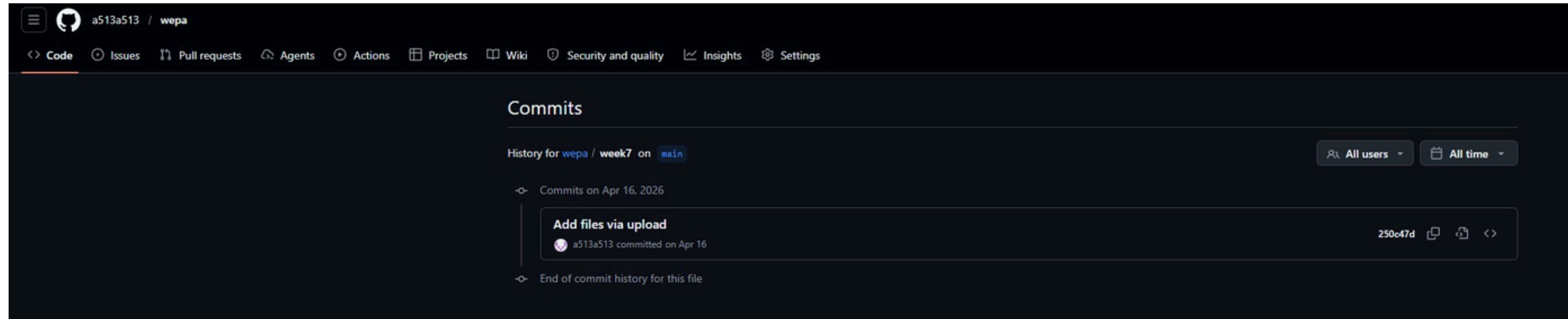
2. 함수형 컴포넌트 (Function Component) 함수형 컴포넌트에서는 `useState`라는 훅(Hook)을 써서 상태를 관리합니다. 괄호 안에 초기값을 넣어주면 [현재 상태 값, 상태를 바꿔주는 함수] 형태의 배열을 반환해 줘서 아주 직관적으로 쓸 수 있어요.

💡 화살표 함수 (Arrow Function)와 람다 화살표 함수는 자바스크립트에서 함수를 아주 짧고 간결하게 쓸 수 있게 해주는 문법(람다 함수)입니다.

작성 예시: `const add = (a, b) => a + b;` (중괄호와 `return`까지 생략 가능!)

가장 큰 특징 (this 바인딩): 일반 함수와 다르게 자기만의 `this`를 새로 만들지 않아요. 대신 자기가 선언된 바깥쪽(Lexical) 스코프의 `this`를 그대로 가져다 쓰기 때문에, 리액트 컴포넌트 안에서 함수를 다룰 때 발생할 수 있는 골치 아픈 `this` 꼬임 문제를 깔끔하게 해결해 줍니다.

7 주차



- Hook -

함수 컴포넌트에서 상태(**state**)와 ****생명주기(lifecycle)**** 기능을 사용하게 해주는 함수입니다.

****생명주기****

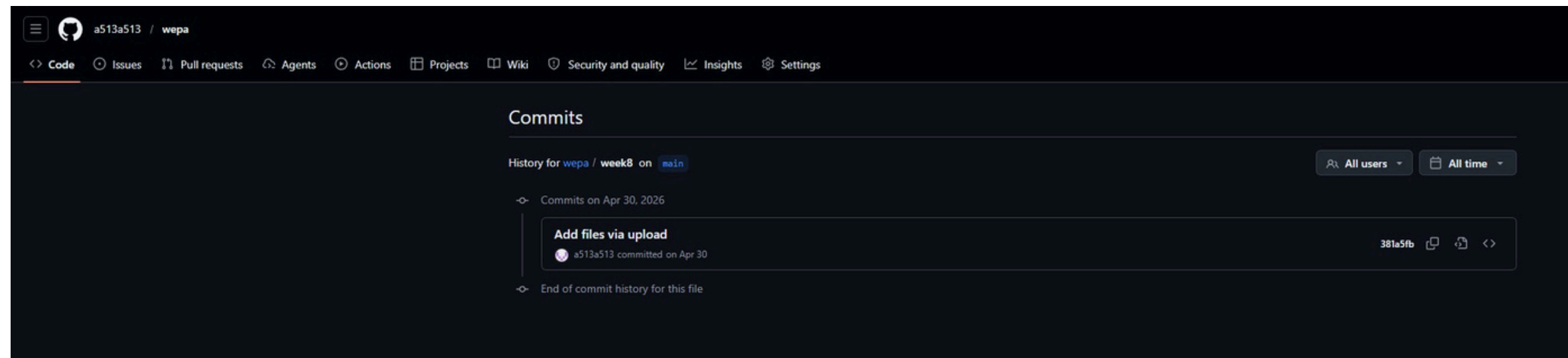
컴포넌트가 생성 -> 업데이트 -> 제거되는 전체 과정

사용 주의 사항

- * **React** 함수 컴포넌트(또는 커스텀 **Hook**)에서만 사용
- * 항상 최상위에서 호출
- * 렌더링 시점에만 호출

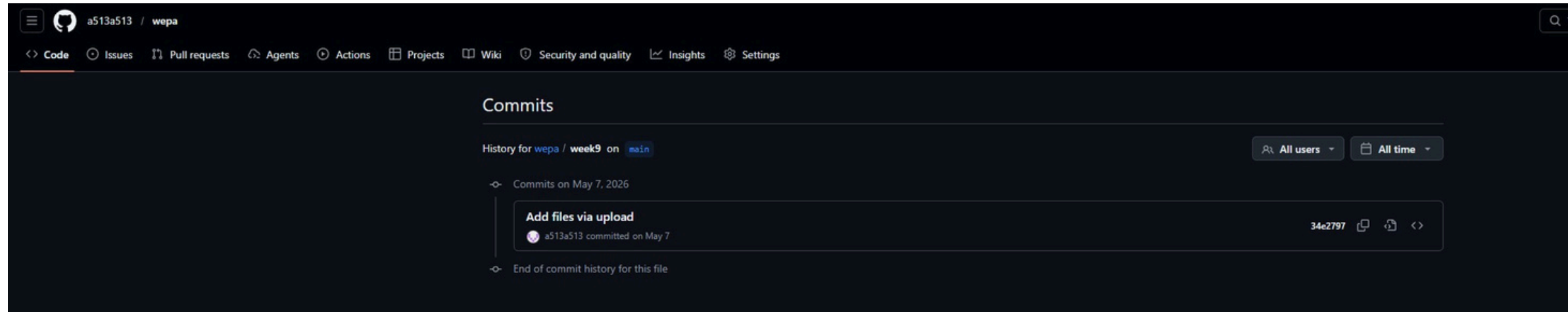
8 주차

팀 프로젝트 준비



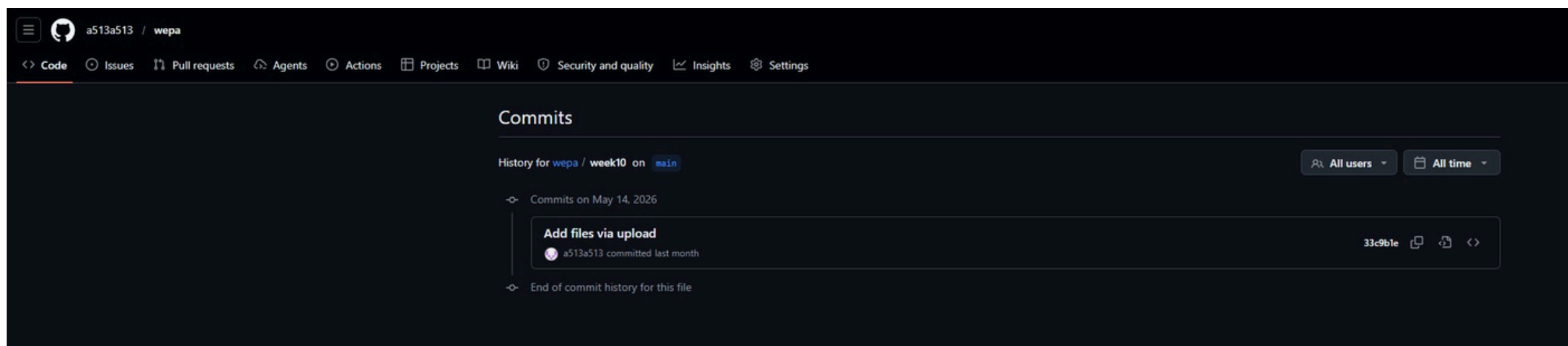
9 주차

팀 프로젝트 준비



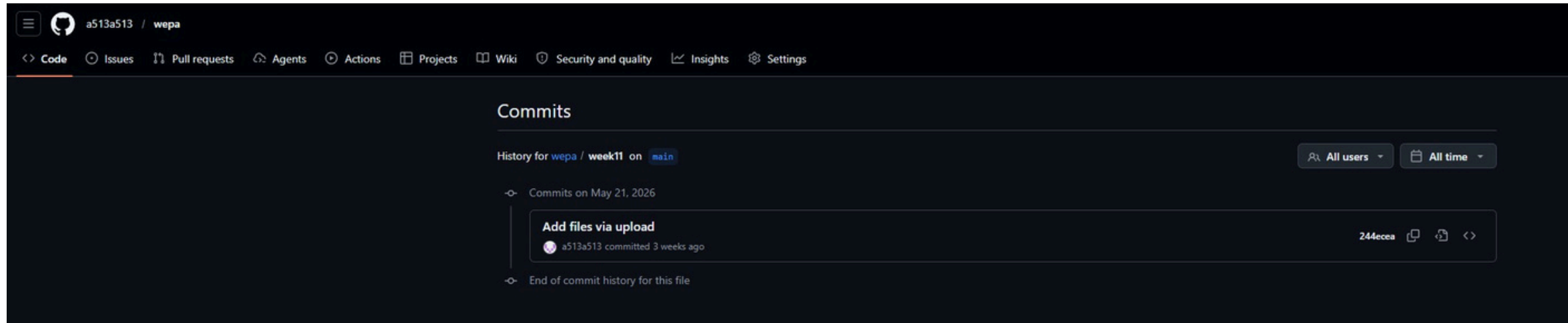
10 주차

다른팀 발표



11 주차

팀 프로젝트 발표



NGINX 기반 GameDrop 웹 서비스 설계 및 배포

웹서비스의 입구, NGINX가 요청을 어떻게 나누는가?



GameDrop 예시로 보는 NGINX의 실제 역할: 사용자 요청을 받아 **React 화면**과 **Express API**로 분배하는 중간 입구 역할



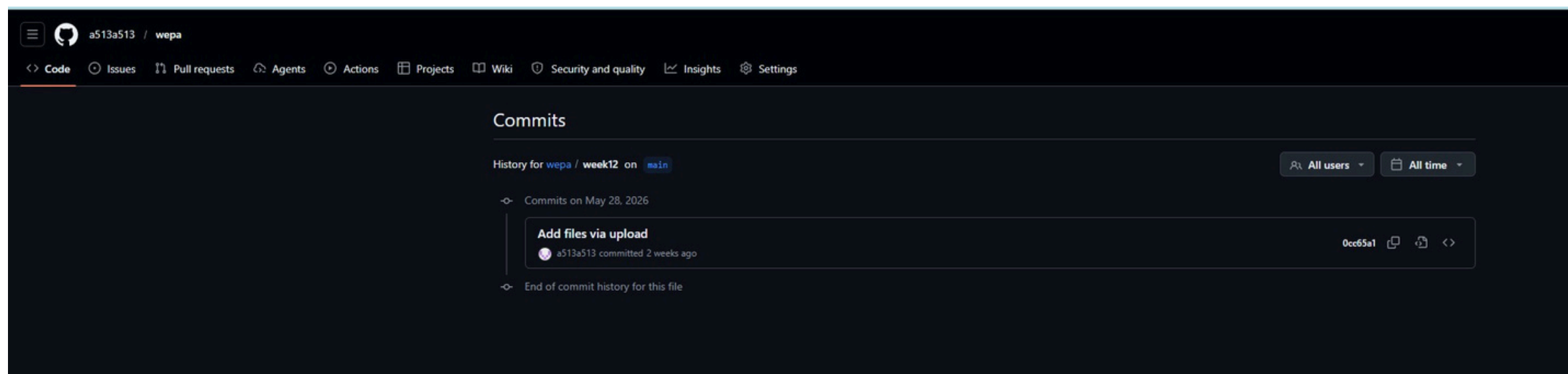
Nginx조 : 김재민(2021663019), 박준홍(2021963034)



2026.05.20

12 주차

다른 팀 발표



총점 : 15

기한에 맞춰서 깃허브에 업로드 하였음